UNIVERSIDAD POLITÉCNICA DE VICTORIA

IMPLEMENTACIÓN DE FLUENTES DE ESTADO MULTIVALUADOS EN MDP-PROBLOG MEDIANTE DISYUNCIONES ANOTADAS.

T E S I N A

QUE PARA OBTENER EL GRADO DE

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN

PRESENTA:

ÁNGEL IVÁN CABRERA ROJAS

DIRECTOR

DR. SAID POLANCO MARTAGÓN

CO-DIRECTOR

DR. HÉCTOR HUGO AVILÉS-ARRIAGA

ORGANISMO RECEPTOR

UNIVERSIDAD POLITÉCNICA DE VICTORIA

CIUDAD VICTORIA, TAMAULIPAS, FEBRERO DE 2026

Ciudad Victoria, Tamaulipas, a **25 de Enero de 2026**

UNIVERSIDAD POLITÉCNICA DE VICTORIA
DR. HÉCTOR HUGO AVILÉS ARRIAGA
PRESENTE

La Universidad Politécnica de Victoria tiene a bien presentar a **CABRERA ROJAS ANGEL IVAN** estudiante del programa académico de **INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN**, con número de matrícula **2230343** y seguro facultativo IMSS número **86-18-04-3681-7;** quién deberá realizar su práctica profesional de **ESTADÍA**, a partir del **07 de Enero de 2026** al **17 de Abril de 2026**, con duración de **600 horas** desarrollando el proyecto **"Implementación de disyunciones anotadas en MDP-ProbLog"** que le fué asignado.

Al concluir se le extenderá la carta de liberación al evaluarlo satisfactoriamente y además le solicitaremos su valiosa opinión respondiendo el formulario que se le enviará por mail, referente al desempeño del practicante, la información que nos proporcione, es de vital importancia para la mejora de los programas académicos que ofrece la Universidad Politécnica de Victoria para la formación de profesionistas altamente especializados.

El practicante deberá cumplir con el Reglamento Interno aplicable al personal en su centro de trabajo.

Sin otro particular.

ATENTAMENTE

OTHÓN CANO GARZA
DIRECTOR DE VINCULACIÓN

C.C.P. SAIDPOLANCOMARTAGÓN
ASESOR INSTITUCIONAL

UNIVERSIDAD POLITÉCNICA DE VICTORIA

Victoria Tamaulipas, a 25 de Enero del 2026
Asunto: Carta de Aceptación

M.A OTHÓN CANO GARZA
DIRECTOR DE VINCULACIÓN
UNIVERSIDAD POLITÉCNICA DE VICTORIA
PRESENTE

Hacemos de su conocimiento que hemos aceptado al estudiante **CABRERA ROJAS ANGEL IVAN** del programa académico de **INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN**, con número de matrícula **2230343** de la Universidad Politécnica de Victoria, para realizar su **ESTADÍA** en nuestra empresa **UNIVERSIDAD POLITÉCNICA DE VICTORIA**, durante el periodo comprendido del día **07 de Enero de 2026** al **17 de Abril de 2026**, el estudiante estará colaborando en el proyecto **"Implementación de disyunciones anotadas en MDP-ProbLog"** con una carga horaria total de **600 horas**

Al concluir satisfactoriamente sus prácticas profesionales, se le entregará al estudiante su carta de liberación debidamente formalizada.

Con el objetivo de colaborar en la mejora de los programas académicos de la Universidad Politécnica de Victoria, estamos de acuerdo en responder a la brevedad posible el formulario de evaluación del desempeño del practicante previo a emitir la liberación de **ESTADÍA**.

Sin otro particular.

ATENTAMENTE
DR. HÉCTOR HUGO AVILÉS ARRIAGA
ASESOR EMPRESARIAL

C.C.P. ARCHIVO

Ing. Juan Camaney de Alba Rojas
PRESENTE

INSERTAR AQUÍ
DOCUMENDO
DE CARTA
DE LIBERACION
DEBIDAMENTE
FIRMADO

CARTA DE ACEPTACIÓN DEL DOCUMENTO PARA SU IMPRESIÓN

Cd. Victoria, Tamaulipas a x

Ángel Iván Cabrera Rojas
PRESENTE

Le comunico que el Programa Académico de Ingeniería en Tecnologías de la Información le ha otorgado la autorización para la impresión de su Tesina de Estadía Práctica cuyo título es:

Implementación de fluentes de estado multivaluados en MDP-ProbLog mediante disyunciones anotadas.

ATENTAMENTE

Dr. Said Polanco Martagón
ASESOR INSTITUCIONAL

c.c.p Director de programa académico

EVALUACIÓN DE ESTADÍA
Rúbrica para evaluación de la presentación y el reporte de estadía

Nombre del alumno: **ÁNGEL IVÁN CABRERA ROJAS** Calificación final: _____

Periodo: **X**

Ponderación	Aspecto a Evaluar	Competente 10	Independiente 9	Básico Avanzado 8	No Competente 5
40	Resultados y Actividades	Estrechamente relacionados al perfil de egreso de su programa académico	Parcialmente relacionados al perfil de egreso de su programa académico	Escasamente relacionados al perfil de egreso de su programa académico	Escasamente relacionados al perfil de egreso de su programa académico
30	Exposición de las actividades de la estadía	Detalladas y sustentadas con respecto a los resultados que se obtuvieron	Detalladas y sustentadas parcialmente con respecto a los resultados que se obtuvieron	Detalladas parcialmente con respecto a los resultados que se obtuvieron	Detalladas escazamente con respecto a los resultados que se obtuvieron
10	Material visual Lenguaje verbal	Uso el lenguaje y la terminología apropiadas; El material visual está organizado, adecuado y suficiente	Uso el lenguaje y la terminología apropiadas El material visual está parcialmente organizado y es suficiente	Uso el lenguaje y la terminología son parcialmente apropiadas; El material visual está parcialmente organizado y es suficiente	Uso el lenguaje y terminología es inapropiado; El material visual no está organizado y es insuficiente
10	Exposición en Idioma Inglés	Pronunciation is clear so language is easily understood (2.5) Uses fluent connected speech, occasionally disrupted by search for correct form of expression (2.5) Uses topic related vocabulary without problems (2.5) Responds to questions using varied and descriptive vocabulary and language structures (2.5)	Pronunciation is understandable, but there are slight errors (2.25) Speech is connected but frequently disrupted by search for correct form of expression (2.25) Uses some topic related vocabulary sufficient to communicate ideas (2.25) Responds to questions using simple but accurate vocabulary and language structures (2.25	Pronunciation is understandable most of the time, marked native accent and many errors (2) Speaks with simple sentences, sometimes not connected, but is understood (2) Uses basic vocabulary to communicate ideas (2) Partly responds to simple questions, with limited vocabulary and language structures (2)	Pronunciation makes language very difficult to understand (1) Uses one-word/two-word utterances (1) Unable to communicate ideas due to lack of vocabulary (1) Uses isolated words or sentence fragments to respond to questions (1)
5	Respuesta a los cuestiona-mientos de los evaluadores	Clara y satisfactoria	Clara y parcialmente satisfactoria	Clara e insuficiente	Confusa e insuficiente
5	Autorización de tesina en tiempo y forma	Presenta en tiempo y forma	Presenta en tiempo y forma con la mayoría de requerimientos solicitados	Presenta en tiempo y con algunas limitantes de los requerimientos solicitados.	Presenta fuera de tiempo y con los mínimos requerimientos solicitados.

Dr. Said Polanco Martagón	X	
ASESOR INSTITUCIONAL	EVALUADOR	EVALUADOR DE INGLÉS

REGISTRO DE EVALUACIÓN DE EXPOSICIÓN DE ESTADÍA

Siendo las X horas del día X, el alumno **Ángel Iván Cabrera Rojas**, del programa académico **Ingeniería en Tecnologías de la Información**, con matricula **2230343**, presentó la exposición de la estadía realizada durante el cuatrimestre **x**, en la **Universidad Politécnica de Victoria**, con el proyecto titulado **Implementación de fluentes de estado multivaluados en MDP-ProbLog mediante disyunciones anotadas.**.

Una vez concluido el proceso de evaluación, y con base a la rúbrica establecida para éste propósito, se determina que la calificación de la estadía es _____.

_____
Dr. Said Polanco Martagón
ASESOR INSTITUCIONAL

_____
X
EVALUADOR

_____

EVALUADOR DE INGLÉS

Agradecimientos

Resumen

La Programación Lógica Probabilística (PLP) es un paradigma que combina la capacidad de representación de la lógica de primer orden con la teoría de la probabilidad para el manejo de la incertidumbre. Este paradigma permite modelar una amplia variedad de dominios que abarcan situaciones de la vida real en donde existen interaccciones complejas entre conjuntos de elementos heterogeneos y distintos niveles de incertidumbre. Dentro de este contexto, destaca el framework MDP-ProbLog, el cual utiliza las capacidades del lenguaje ProbLog para la especificación y resolución de Procesos de Decisión de Markov (MDPs) de horizonte infinito utilizando una representación factorizada, lógica y relacional.

Sin embargo, la arquitectura base de este sistema presenta una limitación estructural: la representación de los fluentes de estado se restringe exclusivamente a variables de Bernoulli independientes (booleanas). Debido a esto, para modelar variables que por naturaleza son multivaluadas, el usuario se ve obligado a utilizar técnicas para descomponerlas en múltiples predicados binarios. En el caso de MDP-ProbLog, esta restricción resulta en un proceso de modelado tedioso, propenso a inconsistencias semánticas y poco natural.

La presente tesina reporta el desarrollo de una extensión al núcleo del framework MDP-ProbLog para solucionar esta problemática. El trabajo abarca la revisión de la arquitectura del código fuente, el planteamiento y diseño de la extensión, la programación, la ejecución de pruebas y su respectiva documentación. Mediante modificaciones a los módulos de gestión de estados y del motor de inferencia, se integró el soporte para fluentes de estado multivaluados utilizando el mecanismo nativo de las Disyunciones Anotadas (Annotated Disjunctions). Como resultado de este desarrollo, el modelado de MDPs que requieren variables con dominios multivaluados se convierte en un proceso semánticamente íntegro, mucho más directo y natural para el usuario.

Palabras clave: Programación Lógica Probabilística, MDP-ProbLog, Disyunciones Anotadas, Fluentes Multivaluados, Espacio de Estados.

Summary

Probabilistic Logic Programming (PLP) is a paradigm that combines the representational power of first-order logic with probability theory to handle uncertainty. This paradigm allows for the modeling of a wide variety of domains encompassing real-world situations where complex interactions exist among sets of heterogeneous elements and varying levels of uncertainty. Within this context, the MDP-ProbLog framework stands out, leveraging the capabilities of the ProbLog language for the specification and resolution of infinite-horizon Markov Decision Processes (MDPs) using a factored, logical, and relational representation.

However, the base architecture of this system presents a structural limitation: the representation of state fluents is restricted exclusively to independent Bernoulli (Boolean) variables. Due to this, to model variables that are naturally multi-valued, the user is forced to use techniques to decompose them into multiple binary predicates. In the case of MDP-ProbLog, this restriction results in a modeling process that is tedious, prone to semantic inconsistencies, and unnatural.

This thesis reports the development of an extension to the core of the MDP-ProbLog framework to address this problem. The work encompasses the review of the source code architecture, the planning and design of the extension, programming, testing, and its respective documentation. Through modifications to the state management modules and the inference engine, support for multi-valued state fluents was integrated using the native mechanism of Annotated Disjunctions. As a result of this development, the modeling of MDPs requiring variables with multi-valued domains becomes a semantically sound, much more direct, and natural process for the user.

Keywords: Probabilistic Logic Programming, MDP-ProbLog, Annotated Disjunctions, Multi-valued Fluents, State Space.

Índice

Agradecimientos vii

Resumen viii

Summary ix

Índice x

1. Introducción 1
- 1.1. Antecedentes . . . 2
- 1.2. Definición del Problema . . . 3
- 1.3. Objetivos . . . 3
  - 1.3.1. Objetivo General . . . 3
  - 1.3.2. Objetivos Específicos . . . 3
- 1.4. Justificación . . . 3
- 1.5. Alcances y Limitaciones . . . 4
  - 1.5.1. Alcances . . . 4
  - 1.5.2. Limitaciones . . . 4

2. Estado del Arte 5
- 2.1. MDPs Factorizados y el Problema de la Representación de Estados . . . 5
- 2.2. Programación Lógica Probabilística para la Resolución de MDPs . . . 6
- 2.3. Disyunciones Anotadas y Compilación de Conocimiento . . . 7
- 2.4. Análisis Comparativo . . . 8
- 2.5. Conclusión del Estado del Arte . . . 9

3. Marco Teórico 10
- 3.1. Programación Lógica . . . 10
  - 3.1.1. Términos, átomos y literales . . . 11
  - 3.1.2. Hechos, reglas y consultas . . . 11
  - 3.1.3. Universo de Herbrand e interpretaciones . . . 11
  - 3.1.4. Resolución SLD y grounding . . . 11
- 3.2. Programación Lógica Probabilística . . . 12
  - 3.2.1. Semántica de distribución . . . 12
  - 3.2.2. Relevancia para MDP-ProbLog . . . 13
  - 3.2.3. El lenguaje ProbLog . . . 14
- 3.3. Conteo de Modelos Ponderados . . . 14
- 3.4. Procesos de Decisión de Markov . . . 15
  - 3.4.1. Definición formal . . . 15
  - 3.4.2. Algoritmo de Iteración de Valor . . . 16
- 3.5. MDPs Factorizados . . . 16
  - 3.5.1. Fluentes de estado . . . 17
- 3.6. El Framework MDP-ProbLog . . . 17
- 3.7. Disyunciones Anotadas . . . 18

3.7.1. Definición formal de LPADs 18
3.7.2. Semántica: selecciones e instancias 19
3.7.3. Exclusión mutua nativa 19
3.7.4. Interpretación causal 20
3.8. Compilación de Conocimiento 21
3.8.1. El mapa de compilación de conocimiento 21
3.8.2. Impacto de la representación en el tamaño del circuito 21

Índice de figuras **23**

Índice de cuadros **24**

Índice de algoritmos **25**

Referencias **26**

1. Introducción

La intersección entre la inteligencia artificial simbólica y los modelos estocásticos ha dado lugar a la Programación Lógica Probabilística, un paradigma que enriquece la capacidad de razonamiento relacional de la lógica de primer orden con el manejo de la incertidumbre inherente a la teoría de probabilidad [1]. En el corazón de este paradigma se encuentra *ProbLog*, un lenguaje que extiende Prolog mediante la introducción de hechos probabilísticos y cuya semántica se basa en la distribución sobre los mundos posibles [2]. ProbLog permite calcular la probabilidad de que una consulta lógica sea verdadera dado un conjunto de evidencias, utilizando técnicas de inferencia como el Conteo de Modelos Ponderados (Weighted Model Counting), transformando el problema de inferencia en una evaluación eficiente sobre circuitos lógicos compilados [3].

Paralelamente, en el campo de la toma de decisiones secuenciales, los Procesos de Decisión de Markov se han establecido como el estándar matemático para modelar agentes que actúan en entornos estocásticos. Formalizados rigurosamente a mediados del siglo XX con los trabajos de Richard Bellman sobre programación dinámica [4], los MDPs proporcionan un marco formal para optimizar una función de utilidad a lo largo del tiempo. Sin embargo, su representación clásica mediante matrices de estados atómicos presenta serias limitaciones de escalabilidad. A medida que aumenta el número de variables que describen el estado del sistema, el tamaño del espacio de estados crece de manera exponencial, un fenómeno acuñado por Bellman como la "maldición de la dimensionalidad" [4]. Esta explosión combinatoria hace que los métodos de resolución clásicos sean computacionalmente intratables para problemas del mundo real con múltiples variables interrelacionadas [5].

Para cerrar la brecha entre la expresividad lógica de ProbLog y la capacidad de decisión de los MDPs, Bueno et al. propusieron el framework **MDP-ProbLog** [6]. Esta herramienta integra los conceptos de estados, acciones y recompensas dentro de la sintaxis de la programación lógica, permitiendo representar MDPs de horizonte infinito de manera relacional y compacta. Al utilizar una representación factorizada, MDP-ProbLog mitiga parcialmente la problemática de la dimensionalidad al descomponer el estado global en un conjunto de características o fluentes. Actualmente, este framework es la base tecnológica del equipo de investigación liderado por el Dr. Héctor Hugo Avilés-Arriaga, quienes lo utilizan para el desarrollo de sistemas de navegación y toma de decisiones en vehículos autónomos.

l presente trabajo aborda el diseño e implementación de fluentes de estado multivaluados en el núcleo de MDP-ProbLog mediante el uso de Disyunciones Anotadas (ADs) [7], con el objetivo de extender las capacidades de representación del sistema, garantizando la integridad semántica de los modelos y sentando las bases para analizar el impacto computacional de esta expresividad.

1.1. Antecedentes

El framework **MDP-ProbLog** fue propuesto en 2016 por Bueno et al. como una extensión del lenguaje ProbLog orientada a representar y resolver Procesos de Decisión de Markov (MDPs) de horizonte infinito [6]. Su principal innovación frente a lenguajes de planificación

contemporáneos (como RDDL o PPDDL) fue la capacidad de aprovechar la programación lógica para modelar dependencias simétricas, transitivas y relaciones cíclicas entre variables.

Para lograr esto, MDP-ProbLog establece convenciones sintácticas específicas que operan sobre la base de ProbLog: los componentes del MDP se definen explícitamente mediante predicados reservados como `state_fluent/1` para variables de estado, `action_fluent/1` para acciones y `utility/2` para utilidades. Por su parte, la dinámica del sistema se modela vinculando el estado actual en el tiempo t con el estado futuro en el tiempo $t + 1$. Para encontrar la política óptima, el *solver* compila el programa en fórmulas proposicionales ponderadas (WMC) y ejecuta el algoritmo clásico de Iteración de Valor.

A pesar de la solidez de sus fundamentos teóricos, la herramienta no ha recibido actualizaciones estructurales en su núcleo de representación desde su publicación hace casi una década. En consecuencia, el framework conserva su diseño original, el cual restringe la definición del mundo a lógica estrictamente binaria; es decir, todos los fluentes de estado definidos deben ser variables de Bernoulli independientes [6].

Actualmente, este framework constituye la base tecnológica del equipo de investigación liderado por el Dr. Héctor Hugo Avilés-Arriaga, quienes lo utilizan para el modelado de toma de decisiones en vehículos autónomos. El uso de MDP-ProbLog les permite contar con modelos explicables y reglas de comportamiento auditables. Sin embargo, debido a la restricción binaria del framework, el proceso de modelado exige una adaptación forzada de la información del entorno.

En la práctica, para que el equipo pueda representar características del entorno que son naturalmente multivaluadas, se requiere realizar un proceso de discretización y binarización en el código. Por ejemplo, para representar la posición de un vehículo en una vía de tres carriles, en lugar de utilizar una variable única `carril(izquierdo|central|derecho)`, el modelador debe declarar tres fluentes booleanos independientes: `carril_izquierdo`, `carril_central` y `carril_derecho`. De igual forma, el estado de un semáforo debe dividirse en predicados separados para sus luces verde, amarilla y roja.

1.2. Definición del Problema

La arquitectura actual del framework MDP-ProbLog carece de un mecanismo nativo para representar y evaluar variables de estado que posean más de dos valores mutuamente excluyentes. Esta restricción a fluentes booleanos independientes genera una desconexión fundamental entre la naturaleza multivaluada de los fenómenos físicos del mundo real y su representación computacional.

Esta limitación delega en el usuario la responsabilidad de mantener la consistencia semántica mediante la escritura exhaustiva de reglas lógicas adicionales. La ausencia de soporte para fluentes multivaluados resulta en un proceso de modelado tedioso y poco natural, aumentando la propensión a errores humanos en la definición de las políticas de decisión y generando un espacio de estados artificialmente inflado con combinaciones semánticamente imposibles.

1.3. Objetivos

1.3.1. Objetivo General

Diseñar e integrar un mecanismo de representación de fluentes de estado multivaluados en el núcleo del framework MDP-ProbLog utilizando Disyunciones Anotadas (Annotated Disjunctions), con el fin de proveer una sintaxis de modelado más natural, asegurar la consistencia semántica en los procesos de decisión estocásticos y evaluar el impacto computacional derivado de esta nueva expresividad.

1.3.2. Objetivos Específicos

- Analizar la arquitectura interna actual de MDP-ProbLog, identificando los módulos críticos encargados de la gestión del espacio de estados y la inferencia probabilística para determinar los puntos de extensión.
- Desarrollar una estructura de datos y adaptar la lógica de instanciación (*grounding*) que permita procesar fluentes multivaluados frente a los fluentes binarios tradicionales.
- Implementar el soporte para Disyunciones Anotadas en la definición de las transiciones de estado, garantizando matemáticamente la exclusión mutua nativa de los valores de una variable sin necesidad de reglas de restricción externas.
- Evaluar el comportamiento de la extensión propuesta, contrastando la simplificación cualitativa del proceso de modelado con las implicaciones cuantitativas generadas en el tamaño de los modelos compilados y los tiempos de inferencia.

1.4. Justificación

La presente tesina representa un paso necesario para la maduración de las herramientas de planificación probabilística. Desde una perspectiva práctica y operativa, la integración de fluentes multivaluados impacta directamente en el flujo de trabajo del equipo de investigación del Dr. Avilés-Arriaga. Al automatizar la exclusión mutua mediante Disyunciones Anotadas, se elimina la necesidad de crear reglas manuales restrictivas, reduciendo drásticamente la carga cognitiva del modelador, acortando los tiempos de desarrollo y mitigando la propensión a errores de diseño.

Desde una perspectiva científica, esta extensión democratiza el uso de la Programación Lógica Probabilística al facilitar su adopción en problemas de ingeniería que requieren representar variables complejas de manera directa. Adicionalmente, al exponer y documentar los desafíos inherentes al crecimiento de los circuitos lógicos generados por las Disyunciones Anotadas, este trabajo sienta un precedente crítico y abre nuevas líneas de investigación enfocadas en la optimización de la compilación de conocimiento para MDPs relacionales.

1.5. Alcances y Limitaciones

1.5.1. Alcances

El proyecto abarca la modificación del núcleo del software MDP-ProbLog, específicamente en los módulos de definición de estados y el puente de comunicación hacia el motor de inferencia. Se incluye el diseño de nuevas estructuras de datos para el manejo de variables multivaluadas, la adaptación de los algoritmos de instanciación y la validación funcional mediante casos de estudio en escenarios de navegación autónoma. El desarrollo garantizará la retrocompatibilidad, asegurando que los modelos booleanos existentes sigan funcionando sin alteraciones.

1.5.2. Limitaciones

El desarrollo se limita a MDPs de horizonte infinito con espacios de estados discretos. Esta investigación se enfoca exclusivamente en la capa de representación y modelado lógico. Por lo tanto, no pretende modificar, optimizar ni reescribir los algoritmos subyacentes de inferencia exacta (Weighted Model Counting) ni los compiladores de circuitos lógicos internos (SDD/d-DNNF) utilizados por ProbLog, los cuales dictan en última instancia la complejidad espacial y temporal de la evaluación. Asimismo, las pruebas funcionales y de rendimiento se realizarán en entornos de simulación y no en vehículos físicos en tiempo real.

2. Estado del Arte

El presente capítulo revisa la literatura científica relacionada con la sustentación técnica de esta tesina: la representación factorizada de MDPs y el problema de las variables de estado, el uso de la Programación Lógica Probabilística como lenguaje de especificación para la toma de decisiones secuenciales, y los mecanismos de exclusión mutua y compilación de conocimiento que permiten la inferencia eficiente. El análisis de estos trabajos permite trazar la trayectoria que conduce al vacío de investigación que la presente tesina aborda: la ausencia de soporte nativo para fluentes de estado multivaluados en frameworks de resolución exacta de MDPs basados en PLP.

2.1. MDPs Factorizados y el Problema de la Representación de Estados

En la formulación clásica de un MDP, el espacio de estados se trata como un conjunto plano donde cada estado es una entidad atómica e indivisible [5]. Esta representación, aunque matemáticamente elegante, se vuelve impracticable cuando el estado del sistema se describe mediante múltiples variables: si un dominio involucra n variables binarias, el número total de estados crece como 2^n, lo que Bellman denominó la *maldición de la dimensionalidad* [4].

Para mitigar este crecimiento exponencial, Boutilier, Dearden y Goldszmidt propusieron la representación factorizada de MDPs, donde las transiciones se codifican mediante Redes Bayesianas Dinámicas (DBNs) y las funciones de valor se manipulan como árboles de decisión [8]. En esta representación, cada variable de estado posee su propia tabla de probabilidad condicional local, y los algoritmos de programación dinámica operan sobre agregaciones de estados en lugar de iterar sobre cada estado individual. Su algoritmo *Structured Policy Iteration* (SPI) demostró que era posible resolver MDPs sin enumerar explícitamente el espacio de estados, siempre que la estructura del problema exhibiera independencias condicionales aprovechables.

En la misma línea, Hoey *et al.* desarrollaron SPUDD, un algoritmo de iteración de valor que reemplaza los árboles de decisión por Diagramas de Decisión Algebraicos (ADDs) [9]. Los ADDs, al ser grafos dirigidos acíclicos en lugar de árboles, permiten compartir subgrafos isomorfos, logrando reducciones de hasta 30 veces en el número de nodos respecto a las representaciones arbóreas. SPUDD demostró la resolución exacta de MDPs con hasta 63 millones de estados.

Sin embargo, tanto SPI como SPUDD comparten una restricción fundamental: operan exclusivamente sobre variables booleanas. Cuando el dominio contiene variables con más de dos valores (por ejemplo, la posición de un vehículo en una vía de tres carriles), el modelador debe descomponer cada variable multivaluada en múltiples variables binarias. Hoey *et al.* reconocen explícitamente esta limitación al señalar que variables ternarias deben expandirse en dos binarias, lo cual agranda el espacio de estados por un factor de 4/3 por cada variable expandida e introduce estados semánticamente imposibles [9]. Este problema de la binarización forzada es precisamente la limitación que la presente tesina aborda en el contexto de

MDP-ProbLog.

En el ámbito de los lenguajes de descripción para planificación, RDDL (*Relational Dynamic Influence Diagram Language*) [10] sí contempla fluentes multivaluados de manera nativa, incluyendo variables enumeradas, enteras y continuas. No obstante, la semántica de RDDL está fundamentada en DBNs proposicionales, lo que impide la representación de dependencias simétricas y transitivas que surgen naturalmente en dominios relacionales. MDP-ProbLog, en contraste, hereda de la programación lógica la capacidad de expresar este tipo de relaciones, pero carece del soporte multivaluado que RDDL ofrece.

2.2. Programación Lógica Probabilística para la Resolución de MDPs

La Programación Lógica Probabilística (PLP) combina la capacidad de representación relacional de la lógica de primer orden con el manejo formal de la incertidumbre mediante la teoría de la probabilidad. De Raedt y Kimmig establecieron los fundamentos unificadores de este paradigma, demostrando cómo la *semántica de distribución* — originalmente propuesta por Sato [2] — permite definir una distribución de probabilidad sobre los mundos posibles de un programa lógico [11]. Bajo esta semántica, cada hecho probabilístico actúa como una variable aleatoria independiente cuya realización determina un programa lógico determinista; la probabilidad de una consulta es entonces la suma de las probabilidades de todos los mundos en los que dicha consulta es verdadera.

ProbLog, el lenguaje que implementa esta semántica, transforma el problema de inferencia probabilística en un problema de Conteo de Modelos Ponderados (WMC) sobre fórmulas booleanas [3, 1]. Esto permite aprovechar técnicas de compilación de conocimiento para lograr inferencia exacta en tiempo polinomial respecto al tamaño del circuito compilado.

La extensión de ProbLog hacia la toma de decisiones se materializó primero en DTProbLog [12], que añadió al lenguaje la capacidad de modelar problemas de decisión episódicos: dado un conjunto de variables de decisión y atributos de utilidad, el sistema encuentra la estrategia que maximiza la utilidad esperada. Sin embargo, DTProbLog está diseñado para problemas de un solo paso y no soporta la resolución de MDPs de horizonte infinito, donde es necesario optimizar una política completa a lo largo del tiempo.

Para cerrar esta brecha, Bueno *et al.* propusieron MDP-ProbLog, que extiende la sintaxis de ProbLog con predicados reservados para declarar fluentes de estado, acciones y utilidades, y emplea programas de distribución de transición de dos pasos temporales para representar la dinámica del MDP [6]. El *solver* implementa iteración de valor sobre el circuito compilado por WMC, demostrando formalmente la equivalencia entre ejecutar el programa probabilístico y realizar un *backup* de Bellman. Una ventaja distintiva de MDP-ProbLog frente a lenguajes como RDDL es su capacidad de representar dependencias cíclicas entre predicados aterrizados — como la influencia mutua entre individuos en un dominio de marketing viral — gracias a la resolución SLD de Prolog [6].

No obstante, la arquitectura de MDP-ProbLog hereda una restricción significativa: todos los fluentes de estado se definen como variables de Bernoulli independientes. Para representar un fluente con k valores posibles, el usuario debe declarar k predicados booleanos y escribir manualmente las reglas de restricción que garanticen que exactamente uno esté activo en cada estado. Esta codificación manual es tediosa, propensa a errores y genera un espacio de estados artificialmente inflado con $2^k - k$ combinaciones semánticamente imposibles.

En una línea alternativa, Nitti, Belle y De Raedt propusieron el uso de Cláusulas Distribucionales Dinámicas (DDC) para modelar MDPs discretos y continuos mediante programación probabilística [13]. Aunque su enfoque es más general al admitir variables continuas, utiliza métodos de muestreo (*importance sampling*) que no garantizan soluciones exactas, a diferencia de la inferencia basada en WMC que emplea MDP-ProbLog.

2.3. Disyunciones Anotadas y Compilación de Conocimiento

El mecanismo que esta tesina propone para resolver la restricción binaria de MDP-ProbLog son las Disyunciones Anotadas (ADs). Vennekens, Verbaeten y Bruynooghe formalizaron los *Logic Programs with Annotated Disjunctions* (LPADs), donde cada cláusula tiene la forma $(h_1 : \alpha_1) \vee \ldots \vee (h_n : \alpha_n) \leftarrow$ cuerpo, con $\sum \alpha_i = 1$ [7]. La semántica se define mediante *selecciones*: para cada cláusula, se elige exactamente un disyunto, produciendo un programa normal cuyo modelo bien fundado determina la interpretación. La probabilidad de una interpretación es la suma de las probabilidades de todas las selecciones que conducen a ella, asumiendo independencia entre las elecciones de distintas cláusulas.

Lo relevante de este formalismo para el presente trabajo es doble. Primero, las ADs garantizan *por construcción* que exactamente un disyunto es verdadero por cada cláusula, lo cual codifica la exclusión mutua de manera nativa sin necesidad de reglas adicionales. Segundo, las ADs admiten una interpretación causal natural: el cuerpo de la cláusula representa una causa, y los disyuntos en la cabeza son sus posibles efectos mutuamente excluyentes, cada uno con su probabilidad asociada [14]. Esta interpretación mapea directamente al concepto de transición estocástica en un MDP: dado un estado y una acción (causa), el siguiente valor de un fluente es uno de varios resultados posibles (efectos).

El trabajo de Vennekens *et al.* demostró además la equivalencia expresiva entre los LPADs y el *Independent Choice Logic* (ICL) de Poole [15] para programas acíclicos. Ambos formalismos comparten la idea de que la incertidumbre se modela mediante elecciones independientes, pero los LPADs ofrecen una sintaxis más natural para representar efectos indeterminados de acciones, ya que permiten colocar las probabilidades directamente en la cabeza de las reglas.

Para que la inferencia sobre programas con ADs sea computacionalmente tratable, ProbLog emplea un *pipeline* de compilación de conocimiento. El programa aterrizado se transforma primero en una fórmula proposicional ponderada, y posteriormente se compila en un circuito lógico. Darwiche y Marquis establecieron la taxonomía de los lenguajes de compilación, identificando que estructuras como las d-DNNFs (*Deterministic Decomposable Negation Normal Forms*) permiten inferencia en tiempo lineal respecto al tamaño del circuito [16]. Más recientemente, Darwiche introdujo los Diagramas de Decisión Sentencial (SDDs), una repre-

sentación canónica que generaliza los BDDs y es estrictamente más sucinta [17]. ProbLog2 adopta los SDDs como su estructura de compilación principal [18], y es sobre estos circuitos donde MDP-ProbLog ejecuta el Conteo de Modelos Ponderados para calcular las probabilidades de transición.

Un aspecto crucial que conecta la compilación de conocimiento con el problema de la representación multivaluada es que el tamaño de los circuitos compilados depende directamente de cómo se formulan las reglas del programa lógico. Una codificación con k hechos booleanos independientes para representar una variable de k valores genera un espacio combinatorio de 2^k configuraciones, de las cuales solo k son semánticamente válidas. En contraste, una AD con k disyuntos expresa la misma variable con exactamente k ramas activas y la restricción $\sum \alpha_i = 1$ ya incorporada en la semántica, lo que potencialmente reduce el tamaño del circuito compilado.

2.4. Análisis Comparativo

La Tabla 1 presenta una comparativa de los principales enfoques revisados en este capítulo. Se analizan diez trabajos representativos agrupados por familia metodológica. Para cada uno se evalúan cuatro dimensiones: el tipo de variables de estado que soporta, el mecanismo de inferencia empleado, la capacidad del lenguaje para expresar relaciones entre entidades, y la forma en que se garantiza la exclusión mutua entre los valores de una misma variable.

Trabajo	Variables de estado	Inferencia	Expresividad relacional	Exclusión mutua
MDPs factorizados proposicionales				
Boutilier *et al.* [8]	Booleanas	Exacta (árboles)	Proposicional (DBNs)	Manual
Boutilier, Dean y Hanks [19]	Booleanas	Exacta (prog. dinámica)	Proposicional	Manual
Hoey *et al.* [9]	Booleanas (binarización)	Exacta (ADDs)	Proposicional (DBNs)	Manual
Lenguajes de planificación				
Sanner [10]	Multival., continuas	Exacta / Aprox.	Relacional (DBNs *lifted*)	Nativa por tipo
Programación Lógica Probabilística				
De Raedt y Kimmig [11]	Booleanas	Exacta (WMC)	Relacional (ProbLog)	No aplica
Fierens *et al.* [3]	Booleanas	Exacta (WMC)	Relacional (ProbLog)	No aplica
Nitti *et al.* [13]	Discretas, continuas	Aprox. (muestreo)	Relacional (DDC)	Implícita
Riguzzi y Swift [20]	Booleanas	Exacta (tabling)	Relacional	No aplica
Exclusión mutua y semántica causal				
Vennekens *et al.* [14]	Multivaluadas (ADs)	Exacta	Relacional	Nativa ($\sum \alpha_i = 1$)
Poole [15]	Booleanas (ICL)	Exacta	Relacional	Nativa (elecciones)
MDP-ProbLog				
Bueno *et al.* [6]	Booleanas	Exacta (WMC/SDDs)	Relacional (ProbLog)	Manual
Propuesta	**Multivaluadas**	**Exacta (WMC/SDDs)**	**Relacional (ProbLog)**	**Nativa (ADs)**

Tabla 1: Análisis comparativo del estado del arte. Se evalúan cuatro dimensiones para cada trabajo: el tipo de variables de estado soportadas, el mecanismo de inferencia, la capacidad relacional del lenguaje, y la garantía de exclusión mutua entre valores de una misma variable.

Como se observa en la tabla, ninguno de los enfoques existentes integra simultáneamente las cuatro capacidades que la propuesta de esta tesina reúne: variables de estado multivaluadas con exclusión mutua nativa, inferencia exacta mediante WMC sobre circuitos compilados, y expresividad relacional completa heredada de la programación lógica.

2.5. Conclusión del Estado del Arte

La revisión de la literatura permite identificar tres hallazgos principales. En primer lugar, la representación factorizada de MDPs ha demostrado ser una estrategia efectiva para mitigar la maldición de la dimensionalidad, permitiendo resolver problemas con millones de estados sin enumeración explícita [8, 9]. Sin embargo, los enfoques proposicionales más consolidados operan exclusivamente sobre variables booleanas, y cuando el dominio requiere variables multivaluadas, la binarización forzada introduce estados semánticamente imposibles y aumenta

artificialmente el tamaño de los circuitos compilados.

En segundo lugar, la Programación Lógica Probabilística ofrece ventajas significativas sobre los formalismos proposicionales al permitir representar relaciones simétricas, transitivas y cíclicas entre las variables del dominio [6, 11]. El framework MDP-ProbLog materializó esta ventaja al combinar la expresividad de ProbLog con la resolución exacta de MDPs de horizonte infinito mediante WMC [3]. No obstante, su arquitectura restringe los fluentes de estado a variables de Bernoulli independientes, lo que limita la naturalidad del proceso de modelado y delega en el usuario la responsabilidad de codificar manualmente las restricciones de exclusión mutua.

En tercer lugar, las Disyunciones Anotadas constituyen un mecanismo formal que garantiza la exclusión mutua de eventos probabilísticos a nivel semántico [14, 15]. Este constructo ya forma parte del lenguaje ProbLog y es utilizado internamente por su motor de inferencia [18], pero su integración en la capa de representación de fluentes de estado dentro de MDP-ProbLog no ha sido explorada ni implementada.

En consecuencia, la presente tesina se posiciona en la intersección de estos tres ejes al proponer la extensión del núcleo de MDP-ProbLog para soportar fluentes de estado multivaluados mediante Disyunciones Anotadas. Esta integración busca preservar las garantías de inferencia exacta del sistema, eliminar la necesidad de binarización manual, y habilitar una representación del espacio de estados que sea tanto más compacta como semánticamente íntegra.

3. Marco Teórico

En este capítulo se establecen los fundamentos teóricos y formales que sustentan la extensión propuesta al *framework* MDP-ProbLog. Inicialmente, se introducen los conceptos básicos de la programación lógica, la semántica de distribución y las características del lenguaje ProbLog. Posteriormente, se formalizan los Procesos de Decisión de Markov (MDPs) y se explica la importancia de su representación factorizada. Finalmente, se presentan las Disyunciones Anotadas como mecanismo de exclusión mutua, así como las técnicas de compilación de conocimiento que hacen posible la inferencia probabilística exacta.

3.1. Programación Lógica

La programación lógica es un paradigma de computación donde los programas se expresan como conjuntos de sentencias lógicas y la ejecución consiste en realizar inferencias sobre dichas sentencias. A diferencia de los lenguajes imperativos, donde el programador especifica *cómo* resolver un problema mediante una secuencia de instrucciones, en la programación lógica se especifica *qué* es verdadero en el dominio, y el motor de inferencia se encarga de derivar las consecuencias.

3.1.1. Términos, átomos y literales

El vocabulario de un programa lógico se construye a partir de tres elementos primitivos: *constantes* (valores concretos como `rojo`, `a`, `3`), *variables* (denotadas con mayúscula inicial, como `X`, `Color`) y *functores* (nombres de predicados o funciones con una aridad asociada). Un *término* es una constante, una variable, o un functor aplicado a una lista de términos. Un *átomo* es un predicado aplicado a sus argumentos, por ejemplo `color(carro, rojo)` o `conectado(a, b)`. Un *literal* es un átomo o su negación.

3.1.2. Hechos, reglas y consultas

Un programa lógico se compone de *cláusulas*, que pueden ser de tres tipos:

- **Hechos**: Afirmaciones incondicionales. Declaran que algo es verdadero sin depender de ninguna condición. Por ejemplo, `padre(tom, bob).` afirma que Tom es padre de Bob.

- **Reglas**: Implicaciones de la forma *cabeza* $\leftarrow$ *cuerpo*. Establecen que la cabeza es verdadera si todas las condiciones del cuerpo se satisfacen. En notación Prolog, esto se escribe como:

  ```
  abuelo(X, Z) :- padre(X, Y), padre(Y, Z).
  ```

  Esta regla establece que `X` es abuelo de `Z` si existe algún `Y` tal que `X` es padre de `Y` e `Y` es padre de `Z`.

- **Consultas**: Preguntas al sistema sobre qué es derivable del programa. Por ejemplo, `?-abuelo(tom, X).` pregunta quiénes son los nietos de Tom.

3.1.3. Universo de Herbrand e interpretaciones

Para asignar significado a un programa lógico, se construye su *universo de Herbrand*: el conjunto de todos los términos sin variables que pueden formarse con las constantes y functores del programa. La *base de Herbrand* es el conjunto de todos los átomos sin variables construibles sobre ese universo. Una *interpretación de Herbrand* es un subconjunto de la base de Herbrand que especifica cuáles átomos son verdaderos. El *modelo mínimo* de un programa definido es la menor interpretación de Herbrand que satisface todas las cláusulas del programa, y constituye la semántica declarativa estándar de la programación lógica.

3.1.4. Resolución SLD y grounding

La ejecución de un programa lógico se basa en el procedimiento de *resolución SLD* (*Selective Linear resolution for Definite clauses*): dado un objetivo (consulta), el sistema busca una cláusula cuya cabeza *unifique* con el objetivo, y reemplaza el objetivo por el cuerpo de la cláusula, aplicando la sustitución de variables correspondiente. Este proceso se repite recursivamente hasta que todos los objetivos se satisfacen (éxito) o no es posible encontrar una unificación (fallo).

El proceso de *grounding* (aterrizado o instanciación) consiste en reemplazar todas las variables de un programa por las constantes del universo de Herbrand, produciendo un programa proposicional equivalente. En el contexto de ProbLog, el grounding es un paso computacional crítico, ya que determina qué porción del programa es relevante para una consulta dada y, por lo tanto, qué tan grande será el circuito lógico resultante.

3.2. Programación Lógica Probabilística

La Programación Lógica Probabilística (PLP) extiende la programación lógica con la capacidad de manejar incertidumbre de manera formal. Mientras que un programa lógico clásico afirma que ciertos hechos son verdaderos de manera determinista, un programa lógico probabilístico permite que los hechos sean verdaderos con una cierta probabilidad.

3.2.1. Semántica de distribución

La base semántica de la PLP fue establecida por Sato mediante la *semántica de distribución* (*distribution semantics*) [2]. Su formulación parte de una observación elegante: si se asigna una distribución de probabilidad a los hechos de un programa lógico, las reglas del programa propagan esa incertidumbre de manera determinista, induciendo una distribución sobre el conjunto de todos los modelos mínimos posibles.

Formalmente, considérese un programa $DB = F \cup R$, donde $F = \{f_1, f_2, \ldots, f_k\}$ es un conjunto de hechos probabilísticos y R es un conjunto de reglas. Cada hecho f_i tiene asociada una probabilidad $\theta_i \in [0, 1]$ y se asume independencia estocástica entre todos los hechos. Una *realización total* (*total choice*) es una asignación de valores de verdad a todos los hechos de F: cada f_i es verdadero o falso. Dado que hay k hechos independientes, existen 2^k realizaciones posibles. La probabilidad de una realización L que incluye los hechos f_i como verdaderos y excluye los restantes es:

$$P(L \mid DB) = \prod_{f_i \in L} \theta_i \prod_{f_i \in F \setminus L} (1 - \theta_i) \tag{1}$$

Cada realización L determina un subconjunto de hechos verdaderos que, combinado con las reglas R, produce un programa lógico definido $L \cup R$ con un modelo mínimo único $M_{DB}(L)$. De esta forma, la distribución sobre realizaciones induce una distribución sobre modelos mínimos. Sato demostró que, bajo condiciones razonables (la *condición de soporte finito*), la masa de probabilidad se concentra exclusivamente en estos modelos derivados: $P_{DB}(\mathrm{fix}(DB)) = 1$ [2].

La probabilidad de éxito (*success probability*) de una consulta q se define como la suma de las probabilidades de todas las realizaciones cuyo modelo mínimo satisface q:

$$P(q \mid DB) = \sum_{L : M_{DB}(L) \models q} P(L \mid DB) \tag{2}$$

Para ilustrar este mecanismo de manera concreta, considérese el siguiente programa mínimo con dos hechos probabilísticos y una regla:

```
0.3::llueve.
0.5::tiene_paraguas.
se_moja :- llueve, \+tiene_paraguas.
query(se_moja).
```

Listing 1: Programa ProbLog mínimo para ilustrar la semántica de distribución.

Este programa define cuatro mundos posibles ($2^2 = 4$ realizaciones de los dos hechos), cada uno con su probabilidad y su modelo mínimo:

`llueve`	`tiene_paraguas`	P(mundo)	`se_moja`	Modelo mínimo
falso	falso	$0.7 \times 0.5 = 0.35$	falso	$\emptyset$
falso	verdadero	$0.7 \times 0.5 = 0.35$	falso	{`tiene_paraguas`}
verdadero	falso	$0.3 \times 0.5 = 0.15$	verdadero	{`llueve`, `se_moja`}
verdadero	verdadero	$0.3 \times 0.5 = 0.15$	falso	{`llueve`, `tiene_paraguas`}

La probabilidad de `se_moja` es la suma de las probabilidades de los mundos donde la consulta es verdadera: $P(\texttt{se_moja}) = 0.15$. Nótese cómo la regla lógica `se_moja :- llueve, \+tiene_paraguas.` propaga la incertidumbre de los hechos hacia la consulta de manera determinista dentro de cada mundo.

3.2.2. Relevancia para MDP-ProbLog

La semántica de distribución es el fundamento teórico sobre el cual opera todo el mecanisno de inferencia de MDP-ProbLog. Cuando el sistema evalúa una transición $P(x'_i = 1 \mid s, a)$, lo que hace internamente es:

1. *Fijar evidencia*: Los fluentes de estado actual y la acción se inyectan como hechos con probabilidad 1.0 (verdadero) o 0.0 (falso), colapsando los mundos posibles a aquellos consistentes con el estado y acción observados.

2. *Calcular la probabilidad de éxito*: El fluente de siguiente estado $x'_i(1)$ se trata como una consulta cuya probabilidad se computa sumando sobre los mundos posibles restantes (los que aún tienen incertidumbre, proveniente de los hechos probabilísticos en las reglas de transición).

De esta manera, la sustitución de evidencia en el circuito compilado es matemáticamente equivalente a calcular la probabilidad condicional $P(x'_i \mid s, a)$ bajo la semántica de distribución, sin necesidad de aplicar explícitamente la regla de Bayes. Esta equivalencia, demostrada formalmente por Bueno *et al.* [6], es lo que permite que cada evaluación del programa ProbLog corresponda a un *backup* de Bellman (Ecuación 5).

3.2.3. El lenguaje ProbLog

ProbLog es un lenguaje de programación lógica probabilística que implementa la semántica de distribución [1, 11]. Su extensión sintáctica respecto a Prolog es mínima: se introduce el concepto de *hecho probabilístico*, denotado por la anotación `p::hecho.`, que indica que el hecho es verdadero con probabilidad p e independiente de todos los demás hechos probabilísticos.

Para ilustrar la sintaxis, el siguiente programa — adaptado del dominio de marketing viral utilizado por Bueno *et al.* [6] y Van den Broeck *et al.* [12] — modela una red social simple donde las personas compran un producto ya sea por marketing directo o por influencia de sus contactos:

```
% Hechos probabilisticos (independientes)
0.3::compra_por_marketing(X) :- persona(X).
0.2::compra_por_influencia(X) :- persona(X).

% Hechos deterministas
persona(ana).  persona(bob).
confia(ana, bob).

% Reglas logicas
compra(X) :- compra_por_marketing(X).
compra(X) :- confia(X, Y), compra(Y),
             compra_por_influencia(X).

% Consulta
query(compra(ana)).
```

Listing 2: Programa ProbLog para un dominio de marketing viral (adaptado de [6]).

En este programa, `compra_por_marketing(X)` es verdadero con probabilidad 0.3 de forma independiente para cada persona `X`. Las reglas determinan las consecuencias lógicas de estas realizaciones. ProbLog calcula $P(\texttt{compra(ana)})$ sumando sobre todos los mundos posibles en los que `compra(ana)` es derivable.

Un aspecto fundamental es que los hechos probabilísticos son estocásticamente independientes entre sí. Esto no es una limitación expresiva, ya que las dependencias entre variables se codifican mediante las reglas lógicas del programa [11].

3.3. Conteo de Modelos Ponderados

El cálculo directo de la Ecuación 2 requiere enumerar exponencialmente muchos mundos posibles, lo cual es computacionalmente intratable en general. ProbLog aborda este problema transformando la inferencia probabilística en un problema de *Conteo de Modelos Ponderados* (WMC, por *Weighted Model Counting*) [3].

El procedimiento consta de tres pasos. Primero, se realiza el *grounding* relevante del programa respecto a la consulta, obteniendo un programa proposicional que contiene solo las cláusulas que pueden influir en el resultado. Segundo, este programa aterrizado se transforma en una fórmula booleana ponderada, donde cada variable proposicional tiene un peso asociado derivado de su probabilidad. Tercero, se calcula el conteo ponderado de los modelos de la fórmula:

$$\mathrm{WMC}(\Delta) = \sum_{\omega \models \Delta} \prod_{l \in \omega} w(l) \tag{3}$$

donde Δ es la fórmula booleana, ω recorre los modelos (asignaciones de verdad que satisfacen Δ), y $w(l)$ es el peso del literal l. Para un hecho probabilístico con probabilidad p, el peso del literal positivo es p y el del literal negativo es $1 - p$.

La probabilidad de la consulta se obtiene entonces como:

$$P(q) = \frac{\mathrm{WMC}(\Delta_q)}{\mathrm{WMC}(\Delta)} \tag{4}$$

donde Δ_q es la fórmula que incorpora la restricción de que q es verdadera. En la práctica, cuando no hay evidencia adicional, el denominador es 1 y la probabilidad se reduce al numerador.

3.4. Procesos de Decisión de Markov

Hasta este punto, se han descrito las herramientas lógicas y probabilísticas para evaluar el estado estático de un sistema. Sin embargo, para modelar agentes que interactúan y modifican su entorno a lo largo del tiempo, es necesario incorporar el paradigma de la toma de decisiones secuenciales.

Un Proceso de Decisión de Markov (MDP) es un modelo matemático para la toma de decisiones secuenciales en entornos estocásticos. En cada paso de tiempo, un agente observa el estado actual del sistema, ejecuta una acción, recibe una recompensa y transita a un nuevo estado según una distribución de probabilidad que depende únicamente del estado actual y la acción ejecutada [5].

3.4.1. Definición formal

Un MDP se define formalmente como una tupla $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, T, R, \gamma \rangle$ donde:

- $\mathcal{S}$ es un conjunto finito de estados completamente observables.
- $\mathcal{A}$ es un conjunto finito de acciones disponibles para el agente.

- $T : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ es la *función de transición*, tal que $T(s, a, s') = P(s' \mid s, a)$ representa la probabilidad de transitar al estado s' dado que el agente está en el estado s y ejecuta la acción a. Esta función satisface la *propiedad de Markov*: el siguiente estado depende únicamente del estado y acción actuales, no del historial previo.

- $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ es la *función de recompensa*, que asigna un valor numérico inmediato a cada par estado-acción.

- $\gamma \in [0, 1)$ es el *factor de descuento*, que pondera la importancia relativa de las recompensas futuras respecto a las inmediatas.

El objetivo de resolver un MDP es encontrar una *política* $\pi : \mathcal{S} \to \mathcal{A}$ que maximice la esperanza de la suma descontada de recompensas futuras. Para MDPs de horizonte infinito con $0 \leq \gamma < 1$, Bellman demostró la existencia de una función de valor óptima V^* que satisface la *ecuación de optimalidad de Bellman* [4]:

$$V^*(s) = \max_{a \in \mathcal{A}} \left[R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' \mid s, a)\, V^*(s') \right] \tag{5}$$

3.4.2. Algoritmo de Iteración de Valor

La *Iteración de Valor* (*Value Iteration*) es un algoritmo de programación dinámica que calcula V^* mediante aproximaciones sucesivas. Partiendo de una función de valor inicial arbitraria $V^{(0)}$, se construye la secuencia:

$$V^{(k+1)}(s) = \max_{a \in \mathcal{A}} \left[R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' \mid s, a)\, V^{(k)}(s') \right] \tag{6}$$

La secuencia $\{V^{(k)}\}$ converge a V^* bajo la norma supremo. Un criterio de convergencia comúnmente utilizado es detenerse cuando el residuo máximo satisface:

$$\max_{s \in \mathcal{S}} \left| V^{(k+1)}(s) - V^{(k)}(s) \right| \leq \frac{2\epsilon(1 - \gamma)}{\gamma} \tag{7}$$

lo cual garantiza que la política *greedy* resultante es ϵ-óptima [**sutton2018reinforcement**, 5].

3.5. MDPs Factorizados

En la formulación clásica, el espacio de estados $\mathcal{S}$ se trata como un conjunto plano de $|\mathcal{S}|$ elementos. Sin embargo, en la mayoría de los dominios prácticos, el estado del sistema se describe naturalmente mediante un conjunto de variables $\mathbf{x} = (x_1, x_2, \ldots, x_n)$, donde cada

variable x_i toma valores en un dominio finito $\mathrm{Dom}(x_i)$. El espacio de estados resultante es el producto cartesiano $\mathcal{S} = \mathrm{Dom}(x_1) \times \mathrm{Dom}(x_2) \times \cdots \times \mathrm{Dom}(x_n)$, cuyo tamaño crece exponencialmente con n.

En un MDP factorizado, la función de transición se descompone aprovechando las independencias condicionales entre las variables de estado. Bajo el supuesto de que el valor futuro de cada variable x'_i depende solo de un subconjunto de las variables actuales y de la acción, la probabilidad de transición se factoriza como [8]:

$$P(s' \mid s, a) = \prod_{i=1}^{n} P(x'_i \mid \mathrm{Pa}(x'_i), a) \tag{8}$$

donde $\mathrm{Pa}(x'_i) \subseteq \{x_1, \ldots, x_n\}$ denota los padres de x'_i en la Red Bayesiana Dinámica (DBN) que codifica las transiciones.

Esta factorización es la que permite a algoritmos como SPUDD [9] y a MDP-ProbLog [6] operar sin enumerar explícitamente todo el espacio de estados.

3.5.1. Fluentes de estado

En el contexto de MDP-ProbLog, las variables de estado se denominan *fluentes* — un término heredado del cálculo de situaciones que denota propiedades del mundo que pueden cambiar a lo largo del tiempo. Cada fluente se estampa temporalmente mediante un argumento adicional: el fluente `vivo` en el tiempo t se representa como `vivo(t)`, y su valor en el siguiente paso temporal como `vivo(t+1)`.

En la arquitectura original de MDP-ProbLog, cada fluente es una variable de Bernoulli que toma valores en $\{0, 1\}$. Para un dominio con n fluentes booleanos, el espacio de estados tiene 2^n estados posibles. La limitación de esta representación se manifiesta cuando el dominio contiene variables con más de dos valores mutuamente excluyentes. Por ejemplo, para representar la posición de un robot en una cuadrícula de tres celdas, se requieren tres fluentes booleanos (`en_celda1`, `en_celda2`, `en_celda3`), generando $2^3 = 8$ combinaciones de las cuales solo 3 son semánticamente válidas. Las 5 combinaciones restantes (el robot en dos celdas simultáneamente, o en ninguna) son estados imposibles que deben excluirse manualmente.

3.6. El Framework MDP-ProbLog

$$\tag{9}$$

MDP-ProbLog es un framework que combina la expresividad de ProbLog con la resolución de MDPs de horizonte infinito [6]. Su sintaxis extiende la de ProbLog con tres tipos de predicados reservados que definen los componentes del MDP. Para ilustrar esto, consideremos un ejemplo clásico de mantenimiento donde un agente debe gestionar el estado de un servidor:

```
% Declaracion de fluentes de estado booleanos
state_fluent(operativo).

% Declaracion de acciones
action(reiniciar).
action(esperar).

% Declaracion de utilidades (recompensas y costos)
utility(operativo(1), 10).    % Recompensa por estar operativo
utility(reiniciar(0), -2).    % Costo penalizado por reiniciar
```

Listing 3: Convenciones sintácticas de MDP-ProbLog para un servidor simple.

La dinámica del sistema se modela mediante reglas lógicas y hechos probabilísticos que vinculan el estado en el instante $t = 0$ (estado actual) con el estado en $t = 1$ (siguiente estado). En la sintaxis de MDP-ProbLog, esto se logra añadiendo el paso de tiempo como argumento a los predicados:

```
% Transicion determinista: reiniciar asegura que el servidor
   funcione
operativo(1) :- reiniciar(0).

% Transicion estocastica: si esta operativo y se espera, puede
   fallar
0.9::mantiene_estado.
operativo(1) :- operativo(0), esperar(0), mantiene_estado.
```

Listing 4: Ejemplo de reglas de transición en MDP-ProbLog.

El solver de MDP-ProbLog implementa el Algoritmo 6 ejecutando los siguientes pasos en cada iteración: (1) para cada par estado-acción, construye un vector de evidencia que fija los valores de los fluentes actuales y la acción; (2) evalúa las probabilidades marginales de los fluentes en $t = 1$ mediante WMC sobre el circuito compilado; (3) calcula la recompensa inmediata y el valor esperado futuro; y (4) actualiza la función de valor. Este proceso se repite hasta alcanzar la convergencia.

La equivalencia formal entre ejecutar el programa probabilístico de MDP-ProbLog y realizar un *backup* de Bellman (Ecuación 5) fue demostrada por Bueno *et al.* [6], garantizando que el solver produce políticas óptimas.

3.7. Disyunciones Anotadas

Las Disyunciones Anotadas (ADs) son un constructo sintáctico y semántico de la PLP que permite representar elecciones probabilísticas entre múltiples alternativas mutuamente excluyentes de manera nativa [7].

3.7.1. Definición formal de LPADs

Un *Logic Program with Annotated Disjunctions* (LPAD) es un conjunto de reglas de la forma:

$$(h_1 : \alpha_1) \vee (h_2 : \alpha_2) \vee \cdots \vee (h_n : \alpha_n) \leftarrow b_1, \ldots, b_m \tag{10}$$

donde los h_i son átomos, los b_j son literales que conforman el cuerpo de la regla, y los $\alpha_i \in [0, 1]$ son probabilidades tales que $\sum_{i=1}^{n} \alpha_i = 1$. Cada regla expresa que, cuando el cuerpo es verdadero, exactamente uno de los disyuntos h_i se hace verdadero, con probabilidad α_i.

En la sintaxis de ProbLog, una AD se escribe de la siguiente manera:

```
% El semaforo cambia de color con cierta probabilidad
0.4::color(verde, 1) ; 0.25::color(amarillo, 1) ;
    0.35::color(rojo, 1) :- color(verde, 0).
```

Listing 5: Ejemplo de Disyunción Anotada en ProbLog.

Esta cláusula establece que si el semáforo está en verde en el tiempo $t = 0$, entonces en $t = 1$ estará en verde con probabilidad 0.4, en amarillo con probabilidad 0.25, o en rojo con probabilidad 0.35. La exclusión mutua (exactamente un color activo) está garantizada por construcción.

3.7.2. Semántica: selecciones e instancias

La semántica de un LPAD se define mediante el concepto de *selección* [7]. Una selección σ es una función que, para cada regla r del programa aterrizado, elige exactamente un par $(h_i : \alpha_i)$ de la cabeza de r. Cada selección produce una *instancia* P_σ: un programa lógico normal (sin disyunciones) donde solo el disyunto seleccionado permanece en la cabeza de cada regla.

La probabilidad de una selección se calcula asumiendo independencia entre las elecciones de distintas reglas:

$$C_\sigma = \prod_{r \in P} \sigma_{\text{prob}}(r) \tag{11}$$

donde $\sigma_{\text{prob}}(r)$ es la probabilidad del disyunto seleccionado por σ para la regla r.

La probabilidad de una interpretación I bajo el programa P es la suma de las probabilidades de todas las selecciones cuya instancia tiene a I como modelo bien fundado:

$$\pi_P^*(I) = \sum_{\sigma \in S(I)} C_\sigma \tag{12}$$

donde $S(I) = \{\sigma \mid \mathrm{WFM}(P_\sigma) = I\}$ es el conjunto de selecciones que conducen a la interpretación I.

3.7.3. Exclusión mutua nativa

La propiedad más relevante de las ADs para este trabajo es que la exclusión mutua entre los disyuntos es una *consecuencia directa de la semántica*, no una restricción que deba imponerse externamente. Dado que cada selección elige exactamente un disyunto por regla, es imposible que dos disyuntos de la misma cláusula sean simultáneamente verdaderos en una misma instancia. Esto contrasta radicalmente con la codificación mediante hechos booleanos independientes, donde la exclusión mutua debe garantizarse mediante reglas lógicas adicionales escritas por el usuario.

Para ilustrar esta diferencia, considérese la representación de una variable con tres valores (por ejemplo, el color de un semáforo) bajo ambos enfoques:

```
% Tres fluentes booleanos independientes
state_fluent(verde).
state_fluent(amarillo).
state_fluent(rojo).

% Restricciones manuales de exclusion mutua
% (el usuario debe escribirlas explicitamente)
falso :- verde(0), amarillo(0).
falso :- verde(0), rojo(0).
falso :- amarillo(0), rojo(0).
falso :- not verde(0), not amarillo(0), not rojo(0).
```

Listing 6: Codificación booleana (MDP-ProbLog original).

```
% Un unico fluente multivaluado con exclusion mutua nativa
state_fluent(semaforo(X)) :- color(X).

1/3::color(verde); 1/3::color(amarillo); 1/3::color(rojo).

% La transicion se expresa como una AD:
0.4::semaforo(verde, 1) ; 0.25::semaforo(amarillo, 1) ;
    0.35::semaforo(rojo, 1) :- semaforo(verde, 0).
% No se requieren restricciones adicionales.
```

Listing 7: Codificación con Disyunciones Anotadas (extensión propuesta).

La codificación booleana genera $2^3 = 8$ combinaciones posibles, de las cuales solo 3 son válidas. La codificación con ADs genera exactamente 3 estados, sin combinaciones adicionales.

3.7.4. Interpretación causal

Vennekens, Denecker y Bruynooghe extendieron la semántica de los LPADs mediante *CP-Logic* (*Causal Probabilistic Logic*), formalizando una interpretación causal de las ADs [14]. Bajo esta interpretación, el cuerpo de cada regla representa una *causa* y los disyuntos en la cabeza son sus posibles *efectos*, cada uno con una probabilidad de ocurrencia asociada. Esta perspectiva causal mapea de manera natural al concepto de transición estocástica en un MDP: dado un estado y una acción (la causa), el siguiente valor de un fluente es uno de varios resultados mutuamente excluyentes (los efectos).

Es importante distinguir entre las probabilidades *causales* de las ADs y las probabilidades *condicionales* clásicas. Como señalan Vennekens *et al.*, si un mismo efecto puede ser producido por múltiples causas no excluyentes, la probabilidad causal de que una causa particular produzca el efecto no coincide con la probabilidad condicional del efecto dado la causa [7]. Las ADs modelan la primera, no la segunda.

3.8. Compilación de Conocimiento

El cálculo del WMC (Ecuación 3) sobre una fórmula booleana arbitraria es un problema #P-completo en general. Sin embargo, si la fórmula se transforma previamente en ciertas representaciones canónicas — conocidas como *lenguajes de compilación* — el conteo ponderado puede realizarse en tiempo polinomial respecto al tamaño de la representación compilada [16].

3.8.1. El mapa de compilación de conocimiento

Darwiche y Marquis clasificaron los principales lenguajes de compilación según sus propiedades de sucintez y las operaciones que soportan en tiempo polinomial [16]. Para la inferencia probabilística mediante WMC, las representaciones más relevantes son:

- **d-DNNF** (*Deterministic Decomposable Negation Normal Form*): Un circuito booleano en forma normal de negación donde los nodos AND tienen hijos que no comparten variables (descomponibilidad) y los nodos OR tienen hijos mutuamente excluyentes (determinismo). Sobre esta estructura, el WMC se computa en un único recorrido ascendente (*bottom-up*) del circuito.

- **SDD** (*Sentential Decision Diagram*): Introducidos por Darwiche [17], los SDDs generalizan los Diagramas de Decisión Binarios Ordenados (OBDDs) al permitir decisiones sobre fórmulas en lugar de variables individuales. Los SDDs son canónicos (dado un *vtree*, la representación es única), soportan todas las operaciones booleanas en tiempo polinomial, y son estrictamente más sucintos que los OBDDs.

ProbLog utiliza los SDDs como su estructura de compilación principal [18]. El flujo de inferencia completo es: programa ProbLog → grounding relevante → fórmula proposicional ponderada → compilación a SDD → WMC sobre el SDD.

3.8.2. Impacto de la representación en el tamaño del circuito

Un aspecto fundamental para este trabajo es que el tamaño del circuito compilado depende no solo de la complejidad intrínseca del problema, sino también de *cómo* se formula el programa lógico. Dos programas que describen el mismo MDP pero con diferentes codificaciones pueden producir circuitos de tamaños muy distintos.

En particular, codificar una variable de k valores mediante k hechos probabilísticos booleanos independientes introduce 2^k posibles asignaciones de verdad en el espacio de búsqueda del compilador, de las cuales solo k son semánticamente válidas. Las restricciones de exclusión mutua, aunque eliminan los modelos inválidos durante la inferencia, no necesariamente reducen el tamaño de la estructura compilada, ya que el compilador puede necesitar representar explícitamente la estructura que descarta esas combinaciones.

En contraste, una AD con k disyuntos codifica directamente las k alternativas válidas con la restricción $\sum \alpha_i = 1$ integrada en la semántica. ProbLog implementa las ADs internamente mediante nodos `choice` que garantizan estructuralmente la exclusión mutua, lo cual tiene el potencial de producir circuitos más compactos al evitar la representación de combinaciones imposibles.

Índice de figuras

Índice de cuadros

1. Análisis comparativo del estado del arte. Se evalúan cuatro dimensiones para cada trabajo: el tipo de variables de estado soportadas, el mecanismo de inferencia, la capacidad relacional del lenguaje, y la garantía de exclusión mutua entre valores de una misma variable. 9

Índice de algoritmos

Referencias

[1] Luc De Raedt, Angelika Kimmig y Hannu Toivonen. "ProbLog: A probabilistic prolog and its application in link discovery". En: *IJCAI*. Vol. 7. 2007, págs. 2462-2467.

[2] Taisuke Sato. "A statistical learning method for logic programs with distribution semantics". En: *Proceedings of the 12th International Conference on Logic Programming (ICLP'95)*. Citeseer, 1995.

[3] Daan Fierens et al. "Inference and learning in probabilistic logic programs using weighted Boolean formulas". En: *Theory and Practice of Logic Programming* 15.3 (2015), págs. 358-401.

[4] Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.

[5] Martin L Puterman. *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, 2014.

[6] Thiago P Bueno et al. "Markov decision processes specified by probabilistic logic programming: Representation and solution". En: *2016 5th Brazilian Conference on Intelligent Systems (BRACIS)*. IEEE. 2016, págs. 337-342.

[7] Joost Vennekens, Sofie Verbaeten y Maurice Bruynooghe. "Logic programs with annotated disjunctions". En: *International Conference on Logic Programming*. Springer, 2004, págs. 431-445.

[8] Craig Boutilier, Richard Dearden y Moisés Goldszmidt. "Stochastic dynamic programming with factored representations". En: *Artificial Intelligence* 121.1-2 (2000), págs. 49-107.

[9] Jesse Hoey et al. "SPUDD: Stochastic Planning using Decision Diagrams". En: *Proceedings of the 15th Conference on Uncertainty in Artificial Intelligence (UAI-99)*. 1999, págs. 279-288.

[10] Scott Sanner. "Relational Dynamic Influence Diagram Language (RDDL): Language Description". Unpublished Manuscript, Australian National University. 2010.

[11] Luc De Raedt y Angelika Kimmig. "Probabilistic (logic) programming concepts". En: *Machine Learning* 100.1 (2015), págs. 5-47.

[12] Guy Van den Broeck et al. "DTProbLog: A Decision-Theoretic Probabilistic Prolog". En: *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence (AAAI-10)*. AAAI Press, 2010, págs. 1217-1222.

[13] Davide Nitti, Vaishak Belle y Luc De Raedt. "Planning in discrete and continuous Markov decision processes by probabilistic programming". En: *Machine Learning* 104.2-3 (2016), págs. 327-342.

[14] Joost Vennekens, Marc Denecker y Maurice Bruynooghe. "CP-logic: A language of causal probabilistic events and its relation to logic programming". En: *Theory and Practice of Logic Programming* 9.3 (2009), págs. 245-308.

[15] David Poole. "The Independent Choice Logic for Modelling Multiple Agents under Uncertainty". En: *Artificial Intelligence* 94.1-2 (1997), págs. 7-56.

[16] Adnan Darwiche y Pierre Marquis. "A knowledge compilation map". En: *Journal of Artificial Intelligence Research* 17 (2002), págs. 229-264.

[17] Adnan Darwiche. "SDD: A New Canonical Representation of Propositional Knowledge Bases". En: *Proceedings of the 22nd International Joint Conference on Artificial Intelligence (IJCAI)*. 2011, págs. 819-826.

[18] Jonas Vlasselaer et al. "ProbLog2: Inference and learning in a probabilistic logic programming language". En: *Theory and Practice of Logic Programming* 16.5-6 (2016), págs. 673-689.

[19] Craig Boutilier, Thomas Dean y Steve Hanks. "Decision-theoretic planning: Structural assumptions and computational leverage". En: *Journal of Artificial Intelligence Research* 11 (1999), págs. 1-94.

[20] Fabrizio Riguzzi y Terrance Swift. "Well-definedness and efficient inference for probabilistic logic programming under the distribution semantics". En: *Theory and Practice of Logic Programming* 13.2 (2013), págs. 279-302.